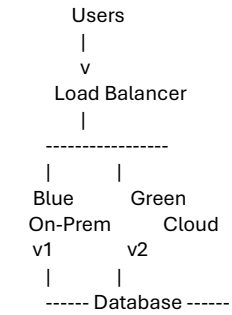
Tell us about a challenging project you worked on and how you overcame obstacles.
Answer:
"I worked on a project where we had to migrate a company's entire system from their on-prem servers to the cloud, which was important for saving costs, but it was challenging because the systems were old and complex. The biggest issue we faced was ensuring that everything worked smoothly without any downtime, as any disruption could affect the company's operations. To solve this, I implemented a blue-green deployment strategy, which allowed us to test the new cloud setup while keeping the old system running. I also automated the entire process using cloud tools, and I trained the team to be more comfortable with the new cloud services. As a result, the migration happened without any issues, and the company saved 40% on infrastructure costs. The team also became much more proficient with cloud technologies."

🔹 **Real-Time Example**
Interviewer: Can you explain exactly what you did during the migration?
Answer:
"First, we analyzed the existing on-prem applications and identified their dependencies, databases, network requirements, and configuration. Then we provisioned the required cloud infrastructure using Terraform. We containerized applications where required using Docker and deployed them in the cloud environment. We kept the existing on-prem environment running while we tested the cloud environment. After functional and performance testing was completed, we gradually moved traffic to the cloud environment using a blue-green approach. We closely monitored the application during the migration and kept the old environment available as a rollback option."
Example architecture:



Initially:
Users → Blue / On-Prem
After validation:
Users → Green / Cloud
If something went wrong:
Users → Blue / On-Prem
So the old environment acted as the rollback option.

🔹 **What exactly was challenging?**
Answer:
"The main challenges were legacy applications, environment differences, dependencies between systems, database migration, and the requirement for minimal or zero downtime. We addressed these by doing the migration in phases, testing everything in a lower environment, automating infrastructure provisioning, and keeping the existing environment available until the cloud environment was fully validated."

🔹 **How did you provision the cloud infrastructure?**
Answer:
"We used Terraform to provision the infrastructure instead of creating resources manually. We created the required networking, compute, security, load balancing, and other resources through reusable Terraform configurations."
Commands:
terraform init
terraform validate
terraform plan
terraform apply
Real-time flow:
Terraform Code
↓
terraform plan
↓
Review Changes
↓
terraform apply
↓
AWS Infrastructure

🔹 **How did you ensure there was no downtime?**
Answer:

“We did not immediately switch all users to the new environment. We first deployed the application in the cloud, tested it, verified health checks and application functionality, and then gradually switched traffic. The old environment remained available so that we could quickly switch traffic back if there was any issue.”

Example:

Before migration:

Users → On-Prem

During migration:

```
Users → On-Prem
        |
        +---- Cloud environment being tested
```

After successful testing:

Users → Cloud

🔹 **How did you test the cloud environment?**

Answer:

“We performed application health checks, functional testing, connectivity testing, performance testing, and monitoring validation. We also verified database connectivity, application logs, CPU and memory usage, and critical business functions before switching production traffic.”

Useful commands:

curl -I https://myapp.example.com

curl -v https://myapp.example.com/health

For Linux:

top

free -m

df -h

🔹 **How did you monitor the migration?**

Answer:

“We monitored application health, CPU, memory, disk, network traffic, error rates, and latency. We used cloud monitoring and application monitoring dashboards to compare the old and new environments.”

For Linux:

top

free -m

df -h

For application logs:

tail -f /var/log/application.log

🔹 **What if the new cloud application failed after traffic was switched?**

Answer:

“Our first priority would be to minimize business impact. We would check the monitoring dashboards and application logs, and if the issue was significant, we would switch traffic back to the previous stable environment. After restoring service, we would investigate the root cause and fix the issue before attempting the migration again.”

Real-time example:

Cloud deployment

↓

Errors increased

↓

Check logs + metrics

↓

Issue not immediately recoverable

↓

Switch traffic back

↓

On-Prem restored

↓

RCA

↓

Fix

↓

Retest

↓

Cloud deployment again

🔥 **Follow-up Questions Interviewer May Ask**

Q1: Why did you choose blue-green deployment?

Answer:

“We chose blue-green deployment because we needed a safe way to migrate without significant downtime. It allowed us to keep the existing environment running while testing the new environment. It also provided a quick rollback option if the new environment had issues.”

Q2: What is blue-green deployment?
Answer:
“Blue-green deployment uses two environments. Blue is the current stable environment and green contains the new version. We deploy and test the new version in green, and once it is validated, we switch traffic from blue to green. If there is a problem, we can switch traffic back to blue.”

Q3: What was your role in the project?
Answer:
“My role was mainly around infrastructure provisioning, deployment automation, CI/CD, cloud migration support, monitoring, and production validation. I worked with developers and other teams to identify dependencies and resolve migration issues.”

Q4: Did you use Docker?
Answer:
“Yes, where containerization was suitable, we containerized applications using Docker. We created Docker images, tested them, pushed them to the container registry, and deployed them in the cloud environment.”
Commands:
docker build -t myapp:1.0 .
docker images
docker run -d -p 8080:8080 myapp:1.0
docker logs <container-id>

Q5: How did you handle database migration?
Answer:
“We first analyzed database dependencies and compatibility. We took appropriate backups and tested the migration in a lower environment. For production, we planned the migration carefully and used a strategy that minimized application downtime. We also validated the data after migration before completing the cutover.”
Example PostgreSQL check:
psql -h <db-host> -U <username> -d <database>
Backup example:
pg_dump -h <db-host> -U <username> <database> > backup.sql

Q6: How did you automate the migration?
Answer:
“We automated infrastructure provisioning using Terraform and deployment activities through CI/CD pipelines. This reduced manual steps and made the migration process repeatable.”
Example:
Git
↓
Jenkins
↓
Terraform Plan
↓
Terraform Apply
↓
Build Docker Image
↓
Push Image
↓
Deploy
↓
Health Check
↓
Monitoring

Q7: How did you handle configuration differences between on-prem and cloud?
Answer:
“We identified environment-specific configurations and externalized them instead of hardcoding them. We managed environment-specific values through configuration management and secrets-management mechanisms and validated them before production cutover.”
Example checks:
env
printenv

Q8: What was the biggest obstacle?
Answer:

“The biggest obstacle was dealing with legacy dependencies and differences between the on-prem and cloud environments. Some applications had assumptions about networking and configuration that were not directly compatible with the cloud environment. We identified these dependencies during assessment, tested them in stages, and resolved them before production cutover.”

Q9: How did you calculate the 40% cost saving?
Answer:
“We compared the previous on-prem infrastructure and operational costs with the cloud infrastructure costs after optimization. We identified unused or over-provisioned resources, right-sized instances, and optimized the infrastructure. Based on the comparison, the overall infrastructure cost was reduced by around 40%.”

🔥 **Very Important Follow-Up**
Interviewer:
“How do you prove that you really worked on this project?”
Don't give only theory.
Say:
“I can explain the migration flow, the infrastructure components, the deployment process, the issues we faced, and the troubleshooting steps. For example, if a cloud application was not reachable, I would check DNS, load balancer health, security groups, routing, application logs, and service health systematically.”
For Linux:
ping <host>
nslookup myapp.example.com
curl -I https://myapp.example.com
For port connectivity:
nc -vz <host> 443

🔥 **Interviewer Can Go One Level Deeper**
If you say:
“We used Terraform.”
They can ask:
“What Terraform commands did you use?”
terraform init
terraform validate
terraform fmt
terraform plan
terraform apply
terraform destroy
If you say:
“We used Docker.”
They can ask:
“How do you troubleshoot a container?”
docker ps
docker ps -a
docker logs <container>
docker inspect <container>
docker exec -it <container> /bin/sh
If you say:
“We used Kubernetes.”
They can ask:
“How do you troubleshoot a pod?”
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
kubectl get svc
kubectl get endpoints
If you say:
“We used AWS.”
They can ask:
“How do you troubleshoot an unreachable EC2?”
You can explain:
EC2 status
↓
System/Instance checks
↓
Security Group
↓
NACL
↓
Route Table
↓
Subnet

↓
Internet/NAT Gateway
↓
OS/service
↓
Application

⭐ **Best 4-Year Experience Version**
If interviewer asks this question directly, **don't give a 5-minute story**.
Give this version:
"I worked on a challenging on-prem to cloud migration project. The main challenge was migrating legacy applications with multiple dependencies while maintaining minimal downtime. I was involved in infrastructure provisioning, deployment automation, monitoring, and production validation. We used Terraform for infrastructure automation and Docker for containerized applications where applicable. We implemented a blue-green approach, keeping the existing environment active while deploying and testing the cloud environment. After validating application health, connectivity, performance, and monitoring, we switched traffic to the cloud environment. We also maintained the old environment as a rollback option. The migration was completed successfully and helped reduce infrastructure costs by around 40%. It also improved our deployment automation and the team's cloud capabilities."